

COD 23-25 LightGBM

Elliot Dean
Office of the Chief Financial Officer | Office of the Assessor
City of Detroit

24 February 2026

Contents

Load Necessary Libraries	1
Load Data	2
Model I	6
Model II	8
Model III	9
Model IV	10
Model V	11
Model VI	12

Load Necessary Libraries

```
library(tidyverse)  # data wrangling & ggplot2
library(magrittr)
library(janitor)
library(kableExtra) # Create unique tables
library(caret)      # train/test split, cross-validation
library(broom)      # tidy model output
library(car)         # VIF, Durbin-Watson
library(lmtest)      # Breusch-Pagan test
library(tidymodels) # collection of packages for modeling and machine learning
library(xgboost)     # Extreme Gradient Boosting
library(bundle)      # Serialize Model Objects with a Consistent Interface
library(data.table)  # fast I/O + data handling
library(lightgbm)    # LightGBM R package
library(pROC)        # AUC
library(here)        # Find files within a project
library(Matrix)      # sparse matrices play nicely with one-hot encoding
library(diffdf)      # tools for working with data frame difference
```

```
# ---- 1. Load data & normalize names ----
DATA_PATH <- read_csv(here("data/city", "salesSFapr2023toMarch2025Updated.csv")) |>
  rename_with(tolower) # clean_names()

OUT_DIR <- here("models/codmods")
```

Load Data

```
load(file = here("data/rdata", "CODsalesSFapr2023Mar2025.RData"))
```

```
OUT_DIR <- here("models/codmods")
```

```
set.seed(42)
```

```
# -----  
# Core: build design matrix from formula  
# -----  
# This is the key AVM-ish piece:  
# - formula defines predictors  
# - model.matrix makes your X  
# - terms + xlevels are saved for consistent future encodes  
#  
make_design <- function(df,  
                        formula,  
                        terms_obj = NULL,  
                        xlevels = NULL,  
                        handle_new_levels = c("na", "drop", "error")) {  
  
  handle_new_levels <- match.arg(handle_new_levels)  
  
  # lock terms (training or reuse at inference)  
  if (is.null(terms_obj)) {  
    terms_obj <- terms(formula, data = df)  
  }  
  
  # --- CRITICAL: pre-sanitize factor columns BEFORE model.frame(xlev=...) ---  
  # If we're using saved xlevels (test/inference), make sure df's factor columns  
  # cannot contain unseen levels, otherwise model.frame will error immediately.  
  if (!is.null(xlevels)) {  
    fac_vars <- intersect(names(xlevels), names(df))  
  
    for (v in fac_vars) {  
      # Work on a temporary vector so we don't mutate df unexpectedly unless needed  
      col <- df[[v]]  
  
      # Ensure it's character so we can safely compare levels/values  
      if (is.factor(col)) col_chr <- as.character(col) else col_chr <- col  
  
      # Only process if it's character/factor-like (ignore numeric-coded cats)  
      if (is.character(col_chr)) {  
        unseen <- !is.na(col_chr) & !(col_chr %in% xlevels[[v]])  
  
        if (any(unseen)) {  
          new_vals <- unique(col_chr[unseen])  
  
          if (handle_new_levels == "error") {  
            stop(sprintf(  
              "New factor levels found in '%s': %s",  
              v, paste(new_vals, collapse = ", ")  
            ))  
          }  
  
          if (handle_new_levels == "drop") {  
            # drop offending rows  
          }  
        }  
      }  
    }  
  }  
}
```

```

    df <- df[!unseen, , drop = FALSE]
    # need to update col_chr after dropping
    col_chr <- df[[v]]
    if (is.factor(col_chr)) col_chr <- as.character(col_chr)
  } else if (handle_new_levels == "na") {
    # convert unseen to NA (LightGBM handles missing well)
    col_chr[unseen] <- NA_character_
  }
}

# Now lock factor levels to training levels
df[[v]] <- factor(col_chr, levels = xlevels[[v]])
}
}

# Build model.frame using locked terms (now safe with xlev)
mf <- model.frame(terms_obj, data = df, na.action = na.pass, xlev = xlevels)

# Capture xlevels during training
if (is.null(xlevels)) {
  xlevels <- .getXlevels(terms_obj, mf)
}

# Response (if present)
y <- model.response(mf)

# Sparse one-hot design matrix
X <- Matrix::sparse.model.matrix(terms_obj, mf)

# Drop intercept if present
if ("(Intercept)" %in% colnames(X)) {
  X <- X[, setdiff(colnames(X), "(Intercept)"), drop = FALSE]
}

list(
  X = X,
  y = y,
  terms = terms_obj,
  xlevels = xlevels,
  feature_names = colnames(X)
)
}

# --- helper: set LightGBM feature names compatibly ---
set_lgb_feature_names <- function(dset, feature_names) {
  # Many builds support setting fields
  if (exists("lgb.Dataset.set.field", where = asNamespace("lightgbm"), inherits = FALSE)) {
    lightgbm::lgb.Dataset.set.field(dset, field_name = "feature_name", data = feature_names)
  }
  dset
}

# -----
# Metrics (regression)
# -----
rmse <- function(y, yhat) sqrt(mean((y - yhat)^2, na.rm = TRUE))
mae <- function(y, yhat) mean(abs(y - yhat), na.rm = TRUE)

```

```

r2 <- function(y, yhat) {
  ss_res <- sum((y - yhat)^2, na.rm = TRUE)
  ss_tot <- sum((y - mean(y, na.rm = TRUE))^2, na.rm = TRUE)
  1 - ss_res/ss_tot
}

# -----
# Train function (AVM regression)
# -----
train_lgbm_avm <- function(df, formula, test_frac = 0.2, handle_new_levels = "na",
  params = list(
    objective = "regression", metric = "rmse", learning_rate = 0.05,
    num_leaves = 256, feature_fraction = 1, bagging_fraction = 1,
    # numleaves=64 feature_fraction = 0.85 bagging_fraction = 0.85
    bagging_freq = 1, min_data_in_leaf = 10, verbosity = -1 #dataleaf = 50
  ), num_boost_round = 10000, early_stopping_rounds = 200) {
  # num_boost_round = 5000 early_stopping_rounds = 100

  # Basic NA strategy: # keep NAs in predictors; model.matrix will keep NA rows,
  # and LightGBM can handle missing values. # But response must not be NA.
  resp_name <- all.vars(formula)[1]
  df <- df[!is.na(df[[resp_name]]), ]

  # Split
  n <- nrow(df) # or df[, .N]
  idx_test <- sample.int(n, size = floor(test_frac * n))
  idx_train <- setdiff(seq_len(n), idx_test)
  train_df <- df[idx_train, ]
  test_df <- df[idx_test, ]

  # Build train design (captures terms + xlevels)
  tr <- make_design(train_df, formula, handle_new_levels = handle_new_levels)

  # Build test design using locked terms + xlevels
  te <- make_design(test_df, formula, terms_obj = tr$terms,
    xlevels = tr$xlevels, handle_new_levels = handle_new_levels)

  # Ensure feature alignment (model.matrix guarantees order if terms identical,
  # but we assert to be safe)
  if (!identical(tr$feature_names, te$feature_names)) {
    # Align test columns to train columns (in case of dropped rows or subtle diffs)
    # Add missing cols in test, drop extra cols.
    missing <- setdiff(tr$feature_names, te$feature_names)
    extra <- setdiff(te$feature_names, tr$feature_names)

    if (length(missing) > 0) {
      add <- Matrix::Matrix(0, nrow = nrow(te$X), ncol = length(missing), sparse = TRUE)
      colnames(add) <- missing
      te$X <- cbind(te$X, add)
    }
    if (length(extra) > 0) {
      te$X <- te$X[, setdiff(colnames(te$X), extra), drop = FALSE]
    }
    # Reorder to match train
    te$X <- te$X[, tr$feature_names, drop = FALSE]
    te$feature_names <- colnames(te$X)
  }
}

```

```

dtrain <- lgb.Dataset(data = tr$X, label = tr$y)
dvalid <- lgb.Dataset(data = te$X, label = te$y)

# ensure column names exist (important for importance tables)
colnames(tr$X) <- tr$feature_names
colnames(te$X) <- tr$feature_names

# set names on datasets (safe no-op if not supported)
dtrain <- set_lgb_feature_names(dtrain, tr$feature_names)
dvalid <- set_lgb_feature_names(dvalid, tr$feature_names)

booster <- lgb.train(
  params = params, data = dtrain, nrounds = num_boost_round, valid = list(valid = dvalid),
  early_stopping_rounds = early_stopping_rounds, eval_freq = 50
)

pred_test <- predict(booster, te$X)

metrics <- list(
  rmse = rmse(te$y, pred_test), mae = mae(te$y, pred_test), r2 = r2(te$y, pred_test)
)

importance <- lgb.importance(booster, percentage = TRUE)

list(
  booster = booster, formula = formula, terms = tr$terms, xlevels = tr$xlevels,
  feature_names = tr$feature_names, metrics = metrics, importance = importance
)
}

```

```

# -----
# Save bundle (model + encoding contract)
# -----
save_avm_bundle <- function(fit, prefix = "lgbm_avm") {
  ts <- stamp()
  model_file <- file.path(OUT_DIR, sprintf("%s_%s.txt", prefix, ts))
  bundle_file <- file.path(OUT_DIR, sprintf("%s_%s_bundle.rds", prefix, ts))
  imp_file <- file.path(OUT_DIR, sprintf("%s_%s_importance.csv", prefix, ts))

  lgb.save(fit$booster, model_file)

  bundle <- list(
    model_file = model_file,
    formula = fit$formula,
    terms = fit$terms,
    xlevels = fit$xlevels,
    feature_names = fit$feature_names,
    best_iter = fit$booster$best_iter
  )
  saveRDS(bundle, bundle_file)

  fwrite(fit$importance, imp_file)

  list(model_file = model_file, bundle_file = bundle_file, imp_file = imp_file)
}

```

```

# -----
# Predict helper (safe inference)

```

```

# -----
# Provide df_new (one row or many); uses saved terms/xlevels to build X in same feature space
predict_avm <- function(bundle_rds, df_new, handle_new_levels = "na") {
  bundle <- readRDS(bundle_rds)
  booster <- lgb.load(bundle$model_file)

  des <- make_design(df_new, bundle$formula,
                    terms_obj = bundle$terms,
                    xlevels = bundle$xlevels,
                    handle_new_levels = handle_new_levels)

  # Align columns to training feature set
  X <- des$X
  if (!identical(colnames(X), bundle$feature_names)) {
    missing <- setdiff(bundle$feature_names, colnames(X))
    extra <- setdiff(colnames(X), bundle$feature_names)

    if (length(missing) > 0) {
      add <- Matrix::Matrix(0, nrow = nrow(X), ncol = length(missing), sparse = TRUE)
      colnames(add) <- missing
      X <- cbind(X, add)
    }
    if (length(extra) > 0) {
      X <- X[, setdiff(colnames(X), extra), drop = FALSE]
    }
    X <- X[, bundle$feature_names, drop = FALSE]
  }

  predict(booster, X)
}

```

```

# -----
# Optional: AVM-ish "newdata_from_sale" helper
# -----
# Given a full df, pull one record by an id and return the row as newdata for predict_avm()
newdata_from_id <- function(df, id_col, id_value) {
  stopifnot(id_col %in% names(df))
  row <- df[get(id_col) == id_value]
  if (nrow(row) == 0) stop("No rows found for that id.")
  row
}

```

Model I

```

# =====
# MAIN EXAMPLE (edit this to match your AVM variables)
# =====
# message("Reading data: ", DATA_PATH)
# df <- fread(DATA_PATH)
df <- dfa

# Example AVM formula: # - choose EXACTLY the variables you want
# - you can include factor() for categorical predictors
# - you can do transforms inside formula (log, poly, interactions, etc.)

```

```
#
# Typical price AVM:
#   ln_price ~ living_area + lot_area + beds + baths + year_built + factor(ecf) + factor(zip) + dist_downtown
#
# If you train on price directly: #   sale_price ~ ...
#
formula_time <- ln_sppsf ~ arterial + single_family1story + half_duplex1story + half_duplex2story +
  smallest + small + large + largest + two_plus_baths + powder_room +
  good + fair + poor + very_poor_unsound + qabove_avg + qbelow_avg +
  sresb_yearbuilt + crawl + slab + ln_garagespaces +
  fireplace + hot_water + elec_baseboard + wall + central_air + months

message("Training LightGBM AVM with formula-driven features...")
mod_time <- train_lgbm_avm(df, formula = formula_time)

message("\nHoldout metrics:")
print(mod_time$metrics)
```

```
## $rmse
## [1] 0.5250003
##
## $mae
## [1] 0.4192726
##
## $r2
## [1] 0.1467276
```

```
message("\nTop 20 features by Gain:")
print(head(mod_time$importance[order(-Gain)], 20))
```

```
##           Feature      Gain      Cover  Frequency
##           <char>      <num>      <num>      <num>
##  1:  sresb_yearbuilt 0.30548162 0.265647841 0.301263617
##  2:      months 0.21582580 0.152790723 0.263965142
##  3:      fair 0.06235920 0.081817140 0.014466231
##  4:    qabove_avg 0.05482179 0.046621414 0.011851852
##  5:  ln_garagespaces 0.05447238 0.056771519 0.078518519
##  6:      poor 0.04065058 0.054295852 0.011764706
##  7:    fireplace 0.03529161 0.046819805 0.039564270
##  8:      large 0.02677543 0.015478933 0.036862745
##  9:    qbelow_avg 0.02571154 0.021474944 0.037821351
## 10: single_family1story 0.02133666 0.015588783 0.031111111
## 11:    central_air 0.01903383 0.046799048 0.019782135
## 12:      good 0.01847376 0.037688476 0.011154684
## 13:      largest 0.01571782 0.020451943 0.015511983
## 14:    powder_room 0.01558844 0.011530946 0.024052288
## 15:    smallest 0.01512384 0.019385406 0.020479303
## 16:    hot_water 0.01408740 0.012115465 0.017342048
## 17:      crawl 0.01387988 0.022321965 0.012984749
## 18:      small 0.01207514 0.009621062 0.021089325
## 19:    two_plus_baths 0.01003477 0.012758768 0.005315904
## 20:  very_poor_unsound 0.00969473 0.024148267 0.005925926
##           Feature      Gain      Cover  Frequency
```

```
# art <- save_avm_bundle(fit)
# message("\nSaved artifacts:")
# print(art)
```

Model II

```
formula_ln_price_no_ecf <- ln_price ~
  ln_landratio + ln_base_bldgsize_ratio +
  arterial + single_family1story + half_duplex1story + half_duplex2story +
  smallest + small + large + largest + two_plus_baths + powder_room +
  good + fair + poor + very_poor_unsound + qabove_avg + qbelow_avg +
  sresb_yearbuilt + crawl + slab + ln_garagespaces +
  fireplace + hot_water + elec_baseboard + wall + central_air + months

message("Training LightGBM AVM with formula-driven features...")
mod_ln_price_no_ecf <- train_lgbm_avm(df, formula = formula_ln_price_no_ecf)

message("\nHoldout metrics:")
print(mod_ln_price_no_ecf$metrics)
```

```
## $rmse
## [1] 0.5178566
##
## $mae
## [1] 0.4175171
##
## $r2
## [1] 0.3491119
```

```
message("\nTop 20 features by Gain:")
print(head(mod_ln_price_no_ecf$importance[order(-Gain)], 20))
```

##	Feature	Gain	Cover	Frequency
##	<char>	<num>	<num>	<num>
## 1:	ln_base_bldgsize_ratio	0.401006066	0.258950896	0.227124183
## 2:	sresb_yearbuilt	0.140078988	0.156976131	0.193899782
## 3:	ln_landratio	0.124444557	0.146244728	0.183877996
## 4:	months	0.107906023	0.094511512	0.176143791
## 5:	qabove_avg	0.038198469	0.012955887	0.003376906
## 6:	fair	0.035319540	0.053568853	0.023148148
## 7:	ln_garagespaces	0.029271161	0.029164849	0.054520697
## 8:	fireplace	0.028201863	0.040755486	0.021078431
## 9:	poor	0.016576990	0.033477692	0.013235294
## 10:	central_air	0.011001377	0.034939555	0.014814815
## 11:	good	0.008797488	0.023232207	0.008442266
## 12:	qbelow_avg	0.008754352	0.013157448	0.016339869
## 13:	single_family1story	0.008414401	0.006922178	0.015740741
## 14:	largest	0.007834162	0.023164258	0.001034858
## 15:	two_plus_baths	0.007435181	0.005925448	0.001797386
## 16:	hot_water	0.006855929	0.008629226	0.011165577
## 17:	crawl	0.004177934	0.009687623	0.005664488
## 18:	very_poor_unsound	0.003785993	0.019552141	0.003867102
## 19:	arterial	0.003416991	0.004716194	0.007135076
## 20:	slab	0.002044017	0.010915949	0.003703704
##	Feature	Gain	Cover	Frequency

```
# art <- save_avm_bundle(fit)
# message("\nSaved artifacts:")
# print(art)
```

Model III

```
formula_base <- ln_price ~
  ln_landratio + ln_base_bldgsize_ratio + arterial +
  single_family1story + half_duplex1story + half_duplex2story + smallest + small + large + largest +
  two_plus_baths + powder_room + good + fair + poor + very_poor_unsound +
  qabove_avg + qbelow_avg + sresb_yearbuilt + crawl + slab + ln_garagespaces +
  fireplace + hot_water + elec_baseboard + wall + central_air + months + ecf

message("Training LightGBM AVM with formula-driven features...")
mod_base <- train_lgbm_avm(df, formula = formula_base)

message("\nHoldout metrics:")
print(mod_base$metrics)
```

```
## $rmse
## [1] 0.4859298
##
## $mae
## [1] 0.3915181
##
## $r2
## [1] 0.4539678
```

```
message("\nTop 20 features by Gain:")
print(head(mod_base$importance[order(-Gain)], 20))
```

##	Feature	Gain	Cover	Frequency
##	<char>	<num>	<num>	<num>
## 1:	ln_base_bldgsize_ratio	0.341311992	0.082781125	0.1871822803
## 2:	sresb_yearbuilt	0.106671581	0.058715219	0.1500726216
## 3:	ln_landratio	0.099440913	0.056353347	0.1304647785
## 4:	months	0.095231941	0.037753688	0.1474219317
## 5:	qabove_avg	0.032814692	0.002819215	0.0021423384
## 6:	fair	0.031003499	0.013746358	0.0188816267
## 7:	fireplace	0.023141419	0.008556695	0.0160493827
## 8:	ln_garagespaces	0.023034890	0.015858736	0.0379811184
## 9:	ecf2R214	0.019837781	0.012475216	0.0079883805
## 10:	poor	0.015298744	0.012436324	0.0079157589
## 11:	good	0.009017846	0.009690622	0.0065722585
## 12:	central_air	0.007534480	0.005678525	0.0103122731
## 13:	ecf3R310	0.007019039	0.012941167	0.0041031227
## 14:	single_family1story	0.007005763	0.002363866	0.0127450980
## 15:	qbelow_avg	0.005957013	0.007718719	0.0095860566
## 16:	hot_water	0.005593474	0.001591459	0.0078794481
## 17:	ecf4R411	0.005068470	0.009969420	0.0058097313
## 18:	ecf7R701	0.004127098	0.006583843	0.0028322440
## 19:	ecf7R702	0.003801674	0.010598820	0.0038489470
## 20:	largest	0.003708373	0.006130709	0.0006899056
##	Feature	Gain	Cover	Frequency

```
# art <- save_avm_bundle(fit)
# message("\nSaved artifacts:")
# print(art)
```

Model IV

```
formula_ln_tasp <- ln_tasp_re ~
  ln_landratio + ln_base_bldgsize_ratio + arterial +
  single_family1story + half_duplex1story + half_duplex2story +
  smallest + small + large + largest + two_plus_baths + powder_room +
  good + fair + poor + very_poor_unsound + qabove_avg + qbelow_avg +
  sresb_yearbuilt + crawl + slab + ln_garagespaces +
  fireplace + hot_water + elec_baseboard + wall + central_air + months + ecf
```

```
message("Training LightGBM AVM with formula-driven features...")
mod_tasp <- train_lgbm_avm(df, formula = formula_ln_tasp)
```

```
message("\nHoldout metrics:")
print(mod_tasp$metrics)
```

```
## $rmse
## [1] 0.4140035
##
## $mae
## [1] 0.3387884
##
## $r2
## [1] 0.460942
```

```
message("\nTop 20 features by Gain:")
print(head(mod_tasp$importance[order(-Gain)], 20))
```

	Feature	Gain	Cover	Frequency
	<char>	<num>	<num>	<num>
## 1:	ln_base_bldgsize_ratio	0.413566793	0.117449169	0.207344029
## 2:	sresb_yearbuilt	0.099954880	0.067804049	0.159358289
## 3:	ln_landratio	0.095378775	0.091537012	0.142602496
## 4:	months	0.085064158	0.038168257	0.171051693
## 5:	qabove_avg	0.040249915	0.002942481	0.002245989
## 6:	fair	0.026799783	0.015451198	0.014866310
## 7:	fireplace	0.025092287	0.009556707	0.016007130
## 8:	ecf2R214	0.011495239	0.010063597	0.006773619
## 9:	central_air	0.010781067	0.007578343	0.015757576
## 10:	ln_garagespaces	0.010566428	0.007540849	0.019500891
## 11:	ecf3R319	0.008246052	0.012443724	0.005276292
## 12:	poor	0.007398263	0.010315800	0.005276292
## 13:	qbelow_avg	0.006791042	0.010593222	0.011622103
## 14:	single_family1story	0.006449536	0.002797633	0.012335116
## 15:	ecf7R701	0.006441950	0.009121127	0.003030303
## 16:	ecf3R310	0.005760754	0.017829823	0.003208556
## 17:	ecf4R402	0.005384395	0.015453166	0.002281640
## 18:	ecf1R126	0.004733846	0.009692285	0.003778966
## 19:	good	0.004699341	0.008893006	0.004705882

```
## 20:          ecf7R702 0.004614485 0.010366551 0.003422460
##           Feature      Gain      Cover      Frequency
```

```
# art <- save_avm_bundle(fit)
# message("\nSaved artifacts:")
# print(art)
```

Model V

```
formula_alt <- ln_tasp_re ~
  ln_landratio + lnsbldsint + arterial + single_family1story + half_duplex1story +
  half_duplex2story + smallest + small + large + largest + two_plus_baths + powder_room +
  good + fair + poor + very_poor_unsound + qabove_avg + qbelow_avg +
  sresb_yearbuilt + crawl + slab + garage + fireplace +
  hot_water + elec_baseboard + wall + central_air + months + ecf
```

```
message("Training LightGBM AVM with formula-driven features...")
mod_alt <- train_lgbm_avm(df, formula = formula_alt)
```

```
message("\nHoldout metrics:")
print(mod_alt$metrics)
```

```
## $rmse
## [1] 0.4104404
##
## $mae
## [1] 0.3375845
##
## $r2
## [1] 0.4748453
```

```
message("\nTop 20 features by Gain:")
print(head(mod_alt$importance[order(-Gain)], 20))
```

```
##           Feature      Gain      Cover      Frequency
##           <char>      <num>      <num>      <num>
## 1:      lnsbldsint 0.407315246 0.135397679 0.205946994
## 2:      sresb_yearbuilt 0.095325828 0.074658567 0.156431803
## 3:      ln_landratio 0.093437474 0.080150403 0.144537815
## 4:      months 0.088640820 0.042114893 0.181383323
## 5:      qabove_avg 0.043256303 0.003263857 0.002240896
## 6:      fireplace 0.030585163 0.018029395 0.013488472
## 7:      fair 0.030247289 0.021187422 0.014565826
## 8:      ecf2R214 0.016897041 0.015317049 0.008446456
## 9:      central_air 0.011331664 0.008391173 0.017065288
## 10:      ecf3R319 0.010576320 0.014125228 0.005860806
## 11:      poor 0.009484123 0.013279775 0.006291747
## 12:      ecf3R310 0.007485887 0.016347600 0.003533721
## 13: single_family1story 0.006727143 0.005855445 0.014134885
## 14:      ecf1R126 0.006190691 0.010446179 0.004567981
## 15:      qbelow_avg 0.005644071 0.013453239 0.010816634
## 16:      ecf7R701 0.005263849 0.011560834 0.003361345
## 17:      good 0.005164776 0.011374755 0.005559147
```

```
## 18:          ecf4R402 0.005047396 0.017780378 0.002456367
## 19:          ecf1R104 0.004513679 0.008434697 0.004740358
## 20:          ecf7R702 0.004167283 0.008731724 0.004223228
##           Feature      Gain      Cover      Frequency
```

```
# art <- save_avm_bundle(fit)
# message("\nSaved artifacts:")
# print(art)
```

Model VI

```
formula_alt2 <- ln_tasp_re ~
  ln_landratio + lnsbldsint + arterial + single_family1story + half_duplex1story +
  half_duplex2story + smallest + small + large + largest + two_plus_baths + powder_room +
  good + fair + poor + very_poor_unsound + qabove_avg + qbelow_avg +
  sresb_yearbuilt + crawl + slab + garage + fireplace +
  hot_water + elec_baseboard + wall + central_air + ecf
```

```
message("Training LightGBM AVM with formula-driven features...")
mod_alt2 <- train_lgbm_avm(df, formula = formula_alt2)
```

```
message("\nHoldout metrics:")
print(mod_alt2$metrics)
```

```
## $rmse
## [1] 0.4218465
##
## $mae
## [1] 0.3484755
##
## $r2
## [1] 0.4483119
```

```
message("\nTop 20 features by Gain:")
print(head(mod_alt2$importance[order(-Gain)], 20))
```

```
##           Feature      Gain      Cover      Frequency
##           <char>      <num>      <num>      <num>
##  1:      lnsbldsint 0.462732870 0.147361399 0.264035741
##  2:      sresb_yearbuilt 0.109510021 0.090531785 0.194986349
##  3:      ln_landratio 0.099770501 0.082552252 0.175428146
##  4:      qabove_avg 0.049078499 0.004731679 0.002928766
##  5:      fair 0.032289530 0.021975904 0.018118640
##  6:      fireplace 0.025933625 0.011152147 0.018515761
##  7:      ecf2R214 0.013011128 0.016844341 0.010126582
##  8:      central_air 0.010202095 0.010582523 0.019806404
##  9:      poor 0.008950638 0.010534486 0.006056093
## 10: single_family1story 0.007978841 0.005364853 0.018962522
## 11:      qbelow_avg 0.007249692 0.008646655 0.016182676
## 12:      ecf3R319 0.007195752 0.009767168 0.006353934
## 13:      ecf3R310 0.007001306 0.020544387 0.003375527
## 14:      ecf7R701 0.006634963 0.012075116 0.003921569
## 15:      ecf4R402 0.006318007 0.017551631 0.002829486
```

```
## 16:          ecf1R126 0.005916277 0.014384488 0.005956813
## 17:          good 0.005653558 0.013502661 0.007346736
## 18:          ecf4R411 0.005403138 0.013420152 0.005311492
## 19:          ecf4R401 0.005323405 0.009752930 0.004715810
## 20:          ecf7R702 0.004944275 0.008533045 0.004219409
##           Feature      Gain      Cover  Frequency
```

```
# art <- save_aum_bundle(fit)
# message("\nSaved artifacts:")
# print(art)
```

```
# -----
# Example prediction on a single sale/parcel
# -----
# Suppose your df has sale_id or parcelno:
# new_row <- newdata_from_id(df, "sale_id", 123456)
# yhat <- predict_aum(art$bundle_file, new_row)
# print(yhat)
```

```
diffdf(dfa, dfc)
```